Міністерство освіти і науки України
Національний університет "Львівська політехніка"
Кафедра ЕОМ



Звіт

з лабораторної роботи № 4

з дисципліни: «АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ СИСТЕМ»

на тему «Створення документації doxygen»

Варіант №21

Виконала:
ст. гр. КІ-404
Хилинська Л. В.
Прийняв:
Федак П. Р

Львів – 2024

Порядок виконання роботи:

1. Add doxygen comments for all public functions, classes, properties, fields...
2. Generate documentation based on doxygen comments
3. Required steps

Варіант завдання:

Student number	Game	config format
21	tik-tac-toe 3x3	INI

Теоретичний матеріал

Doxygen — це потужний інструмент для автоматичної генерації документації до коду з коментарів, написаних у спеціальному форматі. Він підтримує багато мов програмування, таких як C++, C, Java, Python, PHP, і багатьох інших. Основна мета Doxygen — спростити створення документації для програмістів і розробників, забезпечуючи чіткий і структурований формат для опису функцій, класів, методів та інших компонентів коду.

Основні можливості Doxygen:

1. Генерація документації: Doxygen автоматично створює документацію у вигляді HTML, LaTeX, PDF, RTF, мануалів і навіть XML.
2. Коментарі до коду: Doxygen використовує спеціальні коментарі, що дозволяють визначити метадані для класів, функцій, змінних та інших елементів коду. Такі коментарі можуть містити пояснення, приклади використання, інформацію про параметри та повернуті значення.
3. Перехресні посилання: Doxygen дозволяє створювати перехресні посилання між різними частинами документації (наприклад, між класами або методами).
4. Структура документації: Doxygen може генерувати різні типи документації — від списку класів і функцій до подробних описів кожного елемента.
5. Модифікація шаблонів: Інструмент дозволяє налаштувати вигляд і стиль документації за допомогою шаблонів.

Основні теги Doxygen:

- @brief: короткий опис функції чи класу.
- @param: опис параметрів функції.
- @return: опис поверненого значення.
- @see: посилання на інші функції або класи.
- @example: приклад використання функції чи класу.

Виконання роботи:

Опис всіх дій, які були зроблені для додавання підтримки Doxygen у проект

1. Встановлення Doxygen

- Doxygen встановлений на локальному середовищі через пакетний менеджер:

```
sudo apt-get update
sudo apt-get install -y doxygen
```

- У CI/CD YAML-файл додано команду для встановлення Doxygen:

```
- name: Install Doxygen
  run: sudo apt-get install -y doxygen
```

2. Створення файлу Doxyfile.in

- У кореневій директорії проекту створено файл Doxyfile.in з налаштуваннями для Doxygen.

  Основний вміст:

```
PROJECT_NAME           = "Game Server"
OUTPUT_DIRECTORY       = "@CMAKE_CURRENT_SOURCE_DIR@/docs"
EXTRACT_ALL            = YES
EXTRACT_STATIC         = YES
INPUT                  = "@CMAKE_SOURCE_DIR@/include"
RECURSIVE              = YES
GENERATE_HTML          = YES
GENERATE_LATEX         = NO
```

Використано @CMAKE_CURRENT_SOURCE_DIR@ для генерування документації у директорії build/docs.

3. Добавляння коментарів у код

- Додані Doxygen-коментарі до всіх функцій, структур, перерахувань та глобальних змінних. Приклад:

```
/**
 * @brief Parses the configuration file and populates the configuration map.
 *
 * @param filename The name of the configuration file.
 * @param configMap The map to populate with configuration data.
 * @return true if the file was successfully parsed, false otherwise.
 */
bool parseConfigFile(const std::string &filename, std::unordered_map<std::string, std::string>
&configMap);

/**
 * @enum FINISHED

 * @brief Represents the game completion status.
 */
enum class FINISHED {
    NOT_YET = -1, /**< Game is not finished. */
    DRAW,         /**< Game is a draw. */
    WINNER        /**< Game has a winner. */
};
```

4. Підтримка в CMake

- Додано підтримку Doxygen у CMakeLists.txt:

```
find_package(Doxygen REQUIRED)

if (DOXYGEN_FOUND)
    set(DOXYGEN_IN ${CMAKE_CURRENT_SOURCE_DIR}/Doxyfile.in)
    set(DOXYGEN_OUT ${CMAKE_CURRENT_BINARY_DIR}/Doxyfile)

    configure_file(${DOXYGEN_IN} ${DOXYGEN_OUT} @ONLY)

    add_custom_target(doc
        COMMAND ${DOXYGEN_EXECUTABLE} ${DOXYGEN_OUT}
        WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}
        COMMENT "Generating API documentation with Doxygen"
        VERBATIM
    )
else ()
    message("Doxygen needs to be installed to generate the documentation")
endif ()
```

5. Зміни у build.sh

- У build.sh додано окрему функцію для генерування документації:

```
generate_doc() {
    echo "Generating documentation..."
    mkdir -p docs
    cd docs || exit
    doxygen ../Doxyfile
    cd - > /dev/null
}
```

- Додано підтримку опції doc:

```
case "$1" in
    ...
    doc)
        generate_doc
        ;;
esac
```

6. Перевірка результату

- Після виконання скрипта build.sh із аргументом doc документація генерується в директорії docs у корені проекту.
- У CI/CD також додається команда для створення документації під час складання:

```
- name: Generate documentation
  run: cmake --build . --target doc
```

Висновок:У ході виконання завдання було встановлено необхідне програмне забезпечення та створено конфігураційний файл Doxygen.in. У коді додано коментарі для функцій, структур, класів і перерахувань у форматі, сумісному з Doxygen. Генерацію документації інтегровано у файли CMakeLists.txt та скрипт build.sh, що забезпечує зручне створення документації як локально, так і в процесах CI/CD. Для автоматизації в CI/CD було внесено зміни до YAML-файла GitHub Actions. У результаті документація стала доступною в директорії docs як локально, так і під час складання проекту.